MODULE 8 — I/O

Senior SWE Interview Track — Operating Systems

8.1 Blocking vs Non-blocking I/O

1. Why Interviewers Ask This

The vocabulary of every server-architecture discussion. Interviewers check you can define it precisely (it's about the *call*, not the program) before letting you say "epoll".

2. Core Concept

- **Blocking**: the syscall doesn't return until it can do work — `read()` on an empty socket sleeps the thread.
- **Non-blocking** (`O_NONBLOCK`): the syscall returns *immediately*: with data if available, else `EAGAIN/EWOULDBLOCK`. The thread never sleeps in the call — but now *you* must discover readiness (polling or readiness APIs). Blocking/non-blocking describes the syscall behavior; sync/async (8.2) describes who completes the operation.

3. Internal Working

Blocking read on empty socket: kernel puts the task on the socket's wait queue (state S), context-switches away; packet arrival (softirq) fills the receive buffer and wakes the queue. Non-blocking read: kernel checks the buffer, copies what's there or returns EAGAIN — never enqueues the task. Regular *files* are a trap: O_NONBLOCK effectively doesn't apply — file reads always "block" for the disk (why event loops use thread pools or io_uring for file I/O).

4. ASCII Diagram

```
BLOCKING                          NON-BLOCKING
T: read(fd) --------+             T: read(fd) -> EAGAIN (instantly)
   [thread SLEEPS]  | pkt          T: ...do other work...
   data ready ------+             T: read(fd) -> EAGAIN
T: <- returns data                pkt arrives
                                  T: read(fd) -> data
1 thread per waiting fd           1 thread, many fds (needs readiness API)
```

5. Real Production Example

Thread-per-connection blocking servers (classic Tomcat/Apache prefork) vs non-blocking event loops (Nginx, Node.js, Netty, Redis). The blocking model dies at ~10k connections from thread memory + context switches — the C10K problem that birthed the non-blocking ecosystem.

6. Advantages

Blocking: simple linear code, errors in-line, great with few connections or virtual threads. Non-blocking: thousands–millions of connections per thread; no per-connection stack.

7. Trade-offs

Blocking: thread cost per idle connection (~MBs stack + switch overhead). Non-blocking: inversion of control (state machines/callbacks), EAGAIN handling everywhere, *must not run blocking calls on the loop*.

8. Common Mistakes

- Conflating non-blocking with asynchronous (non-blocking read still copies data synchronously when ready).
- Forgetting partial reads/writes: non-blocking `write()` may accept 10 KB of your 1 MB — you must track progress (top interview coding trap).
- Blocking file I/O or DNS lookups inside an event loop (freezes every connection).

9. Performance Implications

Idle blocked thread costs memory, not CPU; 100k idle threads is memory-infeasible → readiness-based single-thread loops handle the same with ~MBs. Busy-polling non-blocking fds without a readiness API burns 100% CPU (hence 8.6–8.9).

10–11. Interview & Follow-ups

- "What happens inside the kernel when read() blocks?" "What does read() return on a non-blocking empty socket? On a closed one?" (EAGAIN vs 0=EOF — know the difference)
- "Why do event loops still use thread pools for files?"

12. Coding/Debugging Scenario

Node.js service freezes 2 s at a time: someone added synchronous file reads (`readFileSync`) on the request path → move to async/worker threads; watchdog the loop lag.

13. Best Practices

Never block the loop; handle EAGAIN + partial I/O rigorously; use blocking code only with cheap threads (virtual threads/goroutines).

14. Practice Questions

1. Write a correct non-blocking "write all N bytes" loop (EAGAIN + partial writes + EPOLLOUT re-arm).
2. Explain each return of non-blocking read(): n>0, 0, −1/EAGAIN, −1/ECONNRESET.

8.2 Synchronous vs Asynchronous I/O

1. Why Interviewers Ask This

Everyone says "async"; few define it. Precise taxonomy (sync-blocking / sync-non-blocking / async) is a fast senior signal, and io_uring makes it current.

2. Core Concept

- **Synchronous**: *your thread* performs the I/O when it happens (even if it waited via epoll first — epoll+read is synchronous non-blocking).
- **Asynchronous**: you *submit* an operation; the kernel completes it in the background and notifies you (completion event); your thread never executes the copy/wait. Readiness model (epoll: "you may now read without blocking") vs completion model (io_uring/IOCP: "your read finished, buffer is full").

3. Internal Working

- POSIX AIO: mostly a library thread pool — historically weak.
- **Linux io_uring**: two shared-memory ring buffers (submission queue SQ, completion queue CQ). App writes SQEs, kernel consumes, executes (inline, async worker, or polled), posts CQEs. Zero syscalls possible in steady state (SQPOLL); batches naturally; covers files *and* sockets (fixing epoll's file blind spot).
- Windows IOCP: completion ports — the original mainstream completion API (.NET, SQL Server).

4. ASCII Diagram

```
SYNC + epoll (readiness):            ASYNC io_uring (completion):
wait: epoll_wait -> "fd 7 readable"  submit: SQE{read fd7 into buf}
you:  read(fd7, buf)  <- your CPU    kernel: does the read itself
      does the copy                  you: reap CQE{res=4096} -> buf full

App --SQ ring--> Kernel --CQ ring--> App     (shared memory, few syscalls)
```

5. Real Production Example

io_uring adoption: high-performance proxies/storage engines (e.g., TigerBeetle, some RocksDB/ScyllaDB-style engines use polling designs; ScyllaDB via Seastar uses its own async stack + AIO/io_uring). IOCP powers Windows server stacks. Netty/libuv are readiness-based with worker pools for files — know which is which.

6. Advantages

Async/completion: batching (many ops per syscall), file I/O without pools, fewer wakeups, kernel-side polling options; the copy overlaps your compute.

7. Trade-offs

Buffer lifetime management (kernel owns your buffer until completion — bugs waiting), harder cancellation, newer/less portable (io_uring: Linux 5.x+, security scrutiny caused some hosts to restrict it), more complex mental model.

8. Common Mistakes

- Calling epoll “asynchronous I/O” (it’s synchronous non-blocking with readiness notification) — precision here scores points.
- Ignoring buffer ownership rules in completion APIs.
- “async/await = OS async” (language async is usually cooperative scheduling atop readiness APIs).

9. Performance Implications

Syscall-bound workloads (many tiny ops): io_uring batching cuts syscalls per op toward 0 → big wins post-Spectre (syscalls got pricier). For large sequential I/O, models converge — the device is the limit.

10–11. Interview & Follow-ups

- “Four-quadrant table: blocking×sync/async with an example each.”
- “Readiness vs completion models?” “What problem does io_uring solve that epoll can’t?” (regular-file async, syscall batching)

12. Coding/Debugging Scenario

Migrating a log-ingestion service from read()-per-line to io_uring with registered buffers: measure syscalls/op drop (strace -c) and CPU reclaim.

13. Best Practices

Default to your platform’s mature readiness stack (epoll/kqueue via libuv/Netty/tokio); reach for io_uring when profiling shows syscall overhead or file-I/O-in-loop pain.

14. Practice Questions

1. Classify: blocking read; O_NONBLOCK read; epoll_wait+read; aio_read; io_uring read; sendfile.
2. Design the I/O model for a video-upload gateway (large streaming writes + many idle keepalives).

8.3 Polling

1. Why Interviewers Ask This

Polling vs interrupts is a fundamental cost trade-off that reappears everywhere (device drivers, spinlocks, message queues, Kubernetes controllers) — interviewers use it to test latency/throughput reasoning.

2. Core Concept

Polling = repeatedly checking state (“is data ready?”) instead of being notified. Busy-wait polling burns CPU for minimal latency; periodic polling saves CPU but adds up to one period of latency.

3. Internal Working

- CPU polling a device: read status register in a loop — sub-µs reaction, one core consumed.
- App-level: loop over non-blocking reads (bad), or sleep-then-check (latency).
- Hybrid reality: **NAPI** in the Linux network stack — interrupt for the first packet, then *switch to polling* while traffic is high (amortizes interrupt cost), back to interrupts when idle. DPDK/SPDK: pure userspace polling, kernel bypass, 100% core burn for µs-scale networking/storage. io_uring SQPOLL/IOPOLL: kernel-thread polling rings.

4. ASCII Diagram

```
Busy poll:   while(!ready){}         latency ~ns-us, CPU 100%
Periodic:    while(1){check; sleep(T)}  latency ~T/2 avg, CPU ~0
Interrupt:   sleep until IRQ         latency ~us (handler+wakeup), CPU 0
NAPI hybrid: IRQ once -> poll while busy -> IRQ when idle
Rule: poll when events are FREQUENT (arrival interval < handling cost),
      interrupt when RARE.
```

5. Real Production Example

HFT/trading NICs and DPDK apps pin cores to poll (latency is money); Redis/Nginx use epoll (event-driven) not polling; Kafka consumers `poll()` in a loop (long-poll with timeout — polling shape, blocking wait inside); Kubernetes controllers reconcile on watch events + periodic resync (hybrid again).

6. Advantages

Lowest possible latency; no interrupt/context-switch overhead per event; predictable (no IRQ jitter); simple.

7. Trade-offs

Burns cores while idle; doesn't scale to many idle sources; energy cost; periodic polling trades latency for CPU linearly.

8. Common Mistakes

- Painting polling as always-bad — at high event rates polling is *cheaper* than interrupts (interrupt storms!).
- Writing accidental busy-loops (checking a queue with no backoff/blocking) — a real CPU-100% incident class.

9. Performance Implications

Break-even: if events arrive every E µs and interrupt overhead is I µs, polling wins when $E \lesssim I \times k$. 10 GbE at line rate ≈ 14.8 Mpps — interrupts can't keep up (livelock) → NAPI/DPDK exist.

10–11. Interview & Follow-ups

- "When is polling better than interrupts?" "What is NAPI and why hybrid?" "Why does DPDK bypass the kernel?"

12. Coding/Debugging Scenario

A worker checking an internal queue in a `while(true){ if(q.empty()) continue; }` loop pegs a core at 100% with zero traffic → replace with condition-variable wait or blocking dequeue.

13. Best Practices

Block or subscribe when idle; poll when saturated; hybrid adaptively (the NAPI pattern is the template); always bound busy-wait with backoff.

14. Practice Questions

1. Compute average latency and CPU cost of 1 ms periodic polling vs interrupts for 10 events/sec vs 1M events/sec.
2. Design the consumption model for an internal job queue at 5 jobs/min vs 50k jobs/sec.

8.4 Interrupts

1. Why Interviewers Ask This

Interrupts are how the OS learns anything happened; top/bottom-half design is a classic depth probe, and softirq/CPU0 saturation is a real prod incident.

2. Core Concept

An interrupt is a hardware signal that preempts the CPU and vectors it to a kernel handler. Types: hardware IRQs (NIC, disk, timer), exceptions/faults (page fault, divide-by-zero — synchronous), software interrupts (syscall entry historically). The timer interrupt is what makes preemptive scheduling possible at all.

3. Internal Working

1. Device raises IRQ (modern: MSI-X message) → interrupt controller (APIC) routes to a CPU.
2. CPU finishes current instruction, saves minimal state, jumps via the interrupt descriptor table to the handler.
3. **Top half**: minimal, interrupts-disabled work — ack device, grab data pointer, schedule deferred work.
4. **Bottom half** (softirq/tasklet/workqueue/NAPI poll): the heavy lifting (e.g., TCP processing) runs later with interrupts enabled.
5. Return restores the preempted context (possibly triggering a reschedule). Affinity: each IRQ can be steered to CPUs (`/proc/irq/*/smp_affinity`); RSS/RPS/RFS spread network processing across cores.

4. ASCII Diagram

```
NIC gets packet -> MSI-X IRQ -> APIC -> CPU3
CPU3: [save ctx] -> top half: ack, queue skb, schedule NAPI  (us, IRQs off)
      [restore ctx]
later: softirq NET_RX on CPU3: TCP/IP processing -> socket buffer
       -> wake blocked reader (Module 8.1 wakeup!)
Watch: %irq/%soft in mpstat; single-queue NIC = one hot CPU.
```

5. Real Production Example

- "CPU0 at 100% softirq, others idle" — all NIC queues/IRQs pinned to one core; fix with multi-queue NIC + irqbalance/affinity + RPS. Common on packet-heavy proxies/LBs.
- Interrupt storms from a faulty device freezing a box; interrupt coalescing settings (ethtool -C) trading latency for CPU.

6. Advantages

Zero CPU cost while idle; µs-scale reaction; the foundation of preemption, timers, and I/O completion.

7. Trade-offs

Per-event overhead (context save, cache pollution); storms under load (→ NAPI); handler constraints (no sleeping in top half); jitter for latency-critical threads (hence IRQ isolation on trading/RT boxes).

8. Common Mistakes

- No top/bottom-half story ("the handler processes the packet" — no; it defers).
- Not knowing page faults are (synchronous) interrupts too — same vector machinery.
- Missing softirq as a triage category (`%soft` in mpstat, `/proc/softirqs`).

9. Performance Implications

Interrupt ≈ µs each including cache damage; at 100k+ events/s per core, coalescing/NAPI mandatory. ksoftirqd running hot = deferred work exceeding budget = the box is network-saturated even if "CPU looks free" elsewhere.

10–11. Interview & Follow-ups

- “Packet arrives → my `read()` returns: full path.” (IRQ → NAPI/softirq → socket buffer → wait-queue wakeup → scheduler → read copies)
- “Why split top/bottom halves?” “What is interrupt coalescing and its trade-off?”

12. Coding/Debugging Scenario

Load balancer drops packets at 40% total CPU: `mpstat -P ALL` shows CPU0 100% %soft → distribute NIC queue IRQs across cores, enable RPS, retest.

13. Best Practices

Balance IRQs on multi-queue NICs; isolate latency-critical cores from IRQs; watch `%irq/%soft` and `/proc/softirqs` deltas in triage.

14. Practice Questions

1. Trace a keystroke and a 10 GbE packet through interrupt handling — what differs?
2. When would you *increase* interrupt coalescing, and what latency cost do you accept?

8.5 DMA (Direct Memory Access)

1. Why Interviewers Ask This

Completes the I/O picture: how bytes actually move without burning CPU, and the substrate of “zero-copy” — a favorite senior topic via sendfile/Kafka.

2. Core Concept

DMA lets devices read/write RAM directly, without the CPU copying byte-by-byte (PIO). CPU’s role shrinks to: set up the transfer (addresses, lengths), get an interrupt on completion. Modern NICs/NVMe are DMA engines with rings of descriptors.

3. Internal Working

- Driver builds descriptor rings (scatter-gather lists of physical addresses); device DMAs payloads into/out of those buffers autonomously; completion raises an IRQ (coalesced).
- **IOMMU** translates/limits device addresses (protection against rogue DMA; enables VM device passthrough).
- **Zero-copy** path (`sendfile`, Kafka): disk –DMA–> page cache –(device DMAs straight from page cache)–> NIC. CPU copies: **zero**. Classic read+write path: 2 DMA + 2 CPU copies + 4 context switches; sendfile: 2 DMA + ~0 CPU copies.

4. ASCII Diagram

```
Classic file->socket:                sendfile (zero-copy):
disk -DMA-> pagecache                disk -DMA-> pagecache
pagecache -CPU copy-> user buf       pagecache --(NIC DMAs directly,
user buf -CPU copy-> socket buf         scatter-gather + headers)--> wire
socket buf -DMA-> NIC
2 DMA + 2 CPU copies                 2 DMA + 0 CPU copies
```

5. Real Production Example

Kafka’s sendfile is the canonical story (serving MBs/s per core of consumer traffic with almost no CPU); Nginx `sendfile on;` for static content; NVMe drives doing millions of IOPS are only possible because DMA + deep queues remove the CPU from the data path; RDMA extends the idea across the network (remote DMA — HPC/storage fabrics).

6. Advantages

CPU freed for compute; enables line-rate I/O; scatter-gather removes buffer-assembly copies; foundation of every zero-copy API.

7. Trade-offs

Setup overhead (descriptors, mappings) — not worth it for tiny transfers; cache-coherence management (device wrote RAM behind the CPU's back); security (needs IOMMU); physical-memory pinning for buffers.

8. Common Mistakes

- "Zero-copy means no copies at all" — DMA transfers remain; *CPU* copies are eliminated.
- Not knowing sendfile can't apply when data must be transformed (TLS used to break it; kernel TLS/kTLS restores it).
- Forgetting DMA needs pinned/physically-addressable memory (interaction with paging).

9. Performance Implications

Copy cost ~ per-GB CPU time (memcpy at ~10 GB/s/core): pushing 40 Gb/s through 2 extra copies eats cores; zero-copy reclaims them. Small messages: syscall/setup dominates, zero-copy irrelevant — batch instead.

10–11. Interview & Follow-ups

- "Explain how Kafka serves consumers with near-zero CPU." "Count the copies in read+write vs sendfile." "What's an IOMMU for?" "When does zero-copy *not* help?" (tiny payloads, transformations, TLS w/o kTLS)

12. Coding/Debugging Scenario

Static-file CDN node CPU-bound at high throughput: perf shows copy_user dominant → enable sendfile/splice path (+kTLS if TLS), CPU drops multi-×.

13. Best Practices

Use sendfile/splice/kTLS for bulk unmodified data; batch small I/O; keep hot buffers aligned/pinned where APIs require.

14. Practice Questions

1. Diagram all copies/DMA for: (a) read()+write() proxying, (b) sendfile, (c) splice between two sockets, (d) io_uring with registered buffers.
2. Your service TLS-encrypts everything — does sendfile help? What changes with kTLS?

8.6 select()

1. Why Interviewers Ask This

The historical baseline of I/O multiplexing; its limitations are the exam — they motivate poll and epoll. "select → poll → epoll evolution" is a canonical interview arc.

2. Core Concept

`select(nfds, readfds, writefds, exceptfds, timeout)`: pass bitmaps of FDs; kernel blocks until any is ready; returns *modified bitmaps* you must scan. One thread watches many FDs → the first escape from thread-per-connection.

3. Internal Working

Per call: copy three bitmaps in, kernel iterates **every FD 0..nfds−1**, attaches to each file's poll wait queue, sleeps; on any event, wakes, re-checks all, copies bitmaps out. You then loop over all FDs testing `FD_ISSET`. Bitmaps are destroyed each call → re-build every iteration.

4. ASCII Diagram

```
fd_set bits: [0..1023]  <- FD_SETSIZE hard ceiling!
loop:
  FD_ZERO/FD_SET(all fds)          O(n) userspace
  select(...)                      O(n) kernel scan + copy in/out
  for fd in all: if FD_ISSET ...   O(n) userspace scan
Cost per event ~ O(n_total_fds)  -> collapses at high n
```

5. Real Production Example

Legacy portable daemons and simple tools; still the only truly universal API (every OS). Any modern server on select at scale is a bug — the 1024 limit has caused real outages when connection counts crept up (silent corruption if fd ≥ FD_SETSIZE is FD_SET!).

6. Advantages

Ubiquitous/portable; fine for a handful of FDs; microsecond timeout precision (timeval).

7. Trade-offs

FD_SETSIZE=1024 hard cap; O(n) per call in kernel *and* user space; bitmap rebuild each call; three copies per call; no extra per-FD data.

8. Common Mistakes

- Not stating the 1024 limit and O(n) scans (the two headline flaws).
- Forgetting the sets are input-output (must rebuild).
- Believing “exceptfds = errors” (it’s OOB data mostly; errors show on read/write).

9. Performance Implications

10k FDs, 100 ready: select touches all 10k three-plus times per loop → CPU melts at ~thousands of FDs; epoll touches only the 100 ready. This asymptotic gap *is* the interview point.

10–11. Interview & Follow-ups

- “Limitations of select?” (rehearse: cap, O(n), rebuild, copies) “Why does the kernel have to scan everything?” (stateless API — kernel keeps no interest set between calls)

12. Coding/Debugging Scenario

Legacy gateway breaks mysteriously above ~1000 connections: fd numbers exceed FD_SETSIZE → memory corruption via FD_SET; migrate to poll/epoll immediately.

13. Best Practices

New code: never select. Reading legacy: know its shape. Cross-platform: use libevent/libuv abstractions.

14. Practice Questions

1. Write the canonical select echo-server loop; annotate every O(n) step.
2. Why can’t select’s API be fixed without becoming… poll/epoll? (stateless bitmaps are the API)

8.7 poll()

1. Why Interviewers Ask This

The intermediate step: fixes select’s cap, keeps its asymptotics — tests whether you understand *which* problem each API solved.

2. Core Concept

`poll(fds[], nfds, timeout)`: array of `pollfd {fd, events, revents}` instead of bitmaps. No FD_SETSIZE limit; input (events) and output (revents) separated → no rebuild; richer event set (POLLIN/OUT/ERR/HUP/RDHUP).

3. Internal Working

Same kernel engine as select: copy the whole array in, iterate all entries, attach to wait queues, sleep, wake, re-scan, copy out. Still **O(n) per call**, still stateless between calls — the kernel forgets your interest set every time.

4. ASCII Diagram

```
struct pollfd fds[] = {{fd:5, events:POLLIN}, {fd:9, events:POLLIN|POLLOUT}}
poll(fds, 2, timeout)
 -> kernel scans all n entries (O(n)), fills revents
 -> you scan all n entries for revents != 0 (O(n))
Fixed vs select: no 1024 cap, no rebuild.  Same: O(n), full copy per call.
```

5. Real Production Example

Portable middleware and moderate-connection tools; `ppoll` for signal-safe waiting. Historically Apache pre-event MPMs. Anything beyond ~1k active FDs moved to epoll/kqueue.

6. Advantages

Unlimited FDs; cleaner API; portable (POSIX everywhere); per-FD event richness.

7. Trade-offs

O(n) scan + O(n) copy per call unchanged → same collapse at scale; array of 12-byte structs copied every call (10k FDs = 120 KB per syscall!).

8. Common Mistakes

- "poll fixed select's performance" — it fixed the *limit and API*, not the asymptotics.
- Not resetting/ignoring revents properly; missing POLLHUP handling (peer close).

9. Performance Implications

Identical O(n) core: fine to ~1k FDs, painful at 10k, impossible at 100k. The measured select→poll→epoll benchmark curve (flat vs flat vs constant-per-event) is worth citing.

10–11. Interview & Follow-ups

- "select vs poll — exactly what changed?" "Why is poll still O(n) and what API change removes that?" (stateful registration → epoll)

12. Coding/Debugging Scenario

Profiling a 5k-connection poll server: 60% CPU inside poll+scan loops with 1% connections active → epoll migration cuts CPU to noise.

13. Best Practices

poll for small-scale portable code; epoll/kqueue beyond ~1k FDs; libuv/libevent to abstract.

14. Practice Questions

1. Compute bytes copied per second by poll with 20k FDs at 1k wakeups/s.
2. Convert a select loop to poll — list every semantic difference you must handle.

8.8 epoll()

1. Why Interviewers Ask This

The engine of Nginx/Redis/Netty/Node and the star of “how do you handle 1M connections?” — the most bankable I/O answer in backend interviews.

2. Core Concept

epoll makes the kernel *remember your interest set*: register FDs once (`epoll_ctl`), then `epoll_wait` returns **only ready FDs**. Cost per wait ∝ number of *ready* events, not registered FDs — O(1)-ish per event vs poll’s O(n).

3. Internal Working

- `epoll_create1` → kernel object: red-black tree (registered FDs) + **ready list**.
- `epoll_ctl(ADD)` → insert into tree, hook a callback into the file’s wait queue.
- Event happens (packet arrives) → the file’s wakeup callback appends it to the ready list (event-driven — no scanning!).
- `epoll_wait` → pop the ready list, copy those events out; sleep only if empty.
- **LT (level-triggered, default)**: fd reported *while* readable — forgiving; may re-report.
- **ET (edge-triggered)**: reported only on *transitions* (empty→data). You must read until EAGAIN or you lose wakeups forever — pairs with O_NONBLOCK mandatorily; fewer wakeups, sharper hazard.
- `EPOLLONESHOT` for multi-threaded dispatch; epoll fd is itself pollable (nestable).

4. ASCII Diagram

```
epoll instance:
  [RB-tree: all 1,000,000 registered fds]
  [ready list: {fd42, fd977}]  <- appended by wakeup callbacks
epoll_wait -> returns 2 events (not 1M scans!)

ET pitfall: 2 packets arrive, you read 1 -> buffer still has data,
            NO new edge -> you sleep forever. ET rule: drain to EAGAIN.
```

5. Real Production Example

Nginx (worker per core, epoll ET), Redis (single-threaded epoll LT — event loop *is* the server), Netty/Java NIO, libuv/Node, HAProxy, Envoy. WhatsApp-scale “millions of connections per box” stories are epoll/kqueue (+tuning) stories.

6. Advantages

O(ready) not O(registered); kernel-resident interest set (no per-call copies); scales to millions of idle connections; ET minimizes wakeups; thread-safe ctl.

7. Trade-offs

Linux-only; ET correctness burden (drain loops, starvation of other fds if one fd streams forever — need fairness budgets); thundering herd when multiple threads wait on one instance (EPOLLEXCLUSIVE helps); regular files always report ready (useless for disk I/O — io_uring’s job).

8. Common Mistakes

- Explaining epoll as “faster poll” without the *stateful registration + ready list* mechanism.
- ET without draining to EAGAIN (the classic stuck-connection bug); ET with blocking fds.
- Assuming epoll helps file reads.

9. Performance Implications

1M idle conns, 1k active: poll ≈ scans 1M entries per wakeup; epoll returns 1k events — orders of magnitude. Syscall count becomes the next bottleneck (→ batching, io_uring). C10K solved; C10M = epoll + affinity + SO_REUSEPORT + NIC tuning.

10–11. Interview & Follow-ups

- “How does epoll achieve O(1) per event? What data structures?” “LT vs ET — semantics, bugs, when each?” “How does Nginx use epoll across workers?” (SO_REUSEPORT / EPOLLEXCLUSIVE vs accept mutex)

12. Coding/Debugging Scenario

An ET-based server occasionally freezes single connections under burst → partial drain bug; add read-until-EAGAIN loop + per-fd fairness cap; connections recover.

13. Best Practices

LT unless you need ET’s wakeup economy; always O_NONBLOCK; drain loops with byte budgets; one epoll per worker thread (shared-nothing) beats shared instances.

14. Practice Questions

1. Build the event-loop skeleton: epoll_create/ctl/wait + accept + echo, LT then ET — list every code change ET forces.
2. Design “1M websockets on one box”: epoll strategy, memory budget per conn, accept distribution, backpressure.

8.9 kqueue()

1. Why Interviewers Ask This

Breadth check (BSD/macOS) and a design-comparison probe: kqueue is widely considered the *cleaner* design — one API for many event types.

2. Core Concept

BSD/macOS’s scalable event API: a **kernel event queue** plus one syscall, `kevent()`, that both registers changes and reaps events. Filters generalize beyond FDs: `EVFILT_READ/WRITE`, `EVFILT_TIMER` (timers), `EVFILT_PROC` (process exit/fork), `EVFILT_SIGNAL`, `EVFILT_VNODE` (file changes — powers fsevents-style watching).

3. Internal Working

`kq = kqueue()`; `kevent(kq, changelist, nchanges, eventlist, nevents, timeout)` — submit registrations *and* collect completions in one call (batching built-in; epoll needs one epoll_ctl syscall per registration). Each kevent = (ident, filter, flags, udata…): per-event user data pointer, EV_CLEAR for edge-triggered per event, EV_ONESHOT. Same O(ready) ready-queue architecture as epoll.

4. ASCII Diagram

```
one call does both:
kevent(kq, [ADD fd7 READ, ADD timer 100ms], 2,   <- register (batched!)
           events_out, 64, timeout)              <- reap
Filters: READ | WRITE | TIMER | SIGNAL | PROC | VNODE | USER
epoll equivalents need: epoll_ctl xN + timerfd + signalfd + pidfd + inotify
```

5. Real Production Example

Nginx/HAProxy/Netty/libuv on FreeBSD/macOS use kqueue automatically; Netflix’s FreeBSD-based Open Connect CDN appliances (famous 100s-of-Gb/s per box) run kqueue+sendfile stacks; macOS dev tooling (file watchers) rides EVFILT_VNODE.

6. Advantages

Unified event model (fds, timers, signals, processes, file changes — one queue, one wait); syscall batching by design; per-event udata (no separate lookup map); clean ET semantics per-event.

7. Trade-offs

Not on Linux (portability splits: epoll vs kqueue vs IOCP → why libuv/libevent exist); fewer tuning knobs in some areas; same regular-file limitation for disk reads.

8. Common Mistakes

- Only knowing “kqueue = BSD epoll” — mention filters/batching/udata to show real breadth.
- Assuming code ports directly (semantics differ: registration model, triggering flags).

9. Performance Implications

Same asymptotics as epoll (O(ready)); batching registration can beat epoll at high churn (many short-lived connections = many ctl calls). In practice both saturate NICs before the API is the limit.

10–11. Interview & Follow-ups

- “Compare epoll and kqueue designs — which is cleaner and why?” “How do you watch a timer + socket + child process in one loop on each platform?” (kqueue natively; Linux: timerfd/signalfd/pidfd all as fds into epoll — note how Linux converges by *making everything an fd*)

12. Coding/Debugging Scenario

Porting a Linux epoll server to macOS for dev laptops: introduce libuv (or a small event-loop shim) instead of hand-porting; verify ET assumptions against EV_CLEAR.

13. Best Practices

Write against an abstraction (libuv/libevent/tokio/Netty) unless you’re building the abstraction; test event-loop code on both platforms in CI.

14. Practice Questions

1. Sketch a kqueue loop handling: 10k sockets, a 1 s heartbeat timer, SIGTERM, and a child-process exit — no extra fds needed.
2. Table: select / poll / epoll / kqueue / IOCP / io_uring — model (readiness/completion), asymptotics, portability, killer feature.

Module 8 Cheat Sheet (one page)

Taxonomy: blocking vs non-blocking = does the *call* sleep; sync vs async = who performs the I/O; readiness (epoll/kqueue: “you may read”) vs completion (io_uring/IOCP: “read done”).

API	Model	Cost/call	Limit	Killer fact
select	readiness	O(n) scan ×3	1024 fds	bitmaps rebuilt every call
poll	readiness	O(n) scan+copy	none	fixed cap, not asymptotics
epoll	readiness	O(ready)	Linux only	RB-tree + ready list; LT vs ET (drain to EAGAIN!)
kqueue	readiness	O(ready)	BSD/macOS	filters: timers/signals/procs too; batched kevent
io_uring	completion	~0 syscalls (rings)	Linux 5.x+	async *file* I/O; SQ/CQ shared rings

Data path: IRQ (top half: ack+defer) → softirq/NAPI (protocol work) → socket buffer → wait-queue wakeup → epoll ready list → your read(). **Polling vs interrupts**: poll when events frequent (NAPI switches adaptively; DPDK always); interrupt when rare. **DMA/zero-copy**: sendfile = disk→pagecache→NIC, 0 CPU copies (Kafka/Nginx); zero-copy ≠ zero transfers; TLS needs kTLS. **Traps**: O_NONBLOCK on files ≈ no-op; ET without drain = frozen connection; blocking call in event loop = frozen server; partial writes must be tracked.

Top Interview Questions

1. Blocking/non-blocking × sync/async — 4-quadrant table with examples.
2. select vs poll vs epoll — evolve the design, state each fix.
3. epoll internals: RB-tree, ready list, callbacks; LT vs ET with the classic ET bug.
4. Packet arrival → read() returns: the full kernel path.
5. How does Kafka/Nginx serve at near-zero CPU? (sendfile/DMA copy math)
6. When is polling better than interrupts? (NAPI/DPDK)
7. What does io_uring add over epoll?
8. Design: 1M concurrent connections on one box.

Common Mistakes (module-wide)

- “epoll is async I/O”; “non-blocking = async”.
- ET without drain-to-EAGAIN; blocking calls on the loop; unhandled partial I/O.
- “poll fixed select’s performance”; forgetting FD_SETSIZE corruption.
- “zero-copy = no copies”; expecting O_NONBLOCK/epoll to help disk files.

Mock Interview (self-test, ~25 min)

1. (Design) Chat backend: 2M idle websockets, 20k msgs/s, 16 cores. Full I/O architecture: API choice, threading, accept strategy, backpressure, file I/O plan.
2. (Depth) Interviewer: “epoll_wait just returned fd 42 readable.” Walk backwards to the electrons: every kernel structure and event that made that true.
3. (Code) Implement ET-mode echo handling: the exact drain loop, EAGAIN, partial write buffering, EPOLLOUT re-arm.
4. (Prod) Proxy at 40% CPU drops packets; mpstat: CPU0 100% %soft. Diagnose and fix (IRQ affinity/RSS/NAPI story).
5. (Trap) “We made all our file reads non-blocking with O_NONBLOCK so the loop never stalls.” What actually happened, and name two real fixes (thread pool, io_uring).